Universidad Nacional Autó
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.10

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 21/09/2026

**UNAM no guarda relación alguna con las marcas mencionadas como ejemplo. Las marcas son propiedad de sus titulares conforme a la legislación aplicable, se utilizan con fines académicos y didácticos, por lo que no existen fines de lucro, relación publicitaria o de patrocinio.

```
import numpy as np
import matplotlib.pyplot as plt
from random import randint
from time import sleep
from IPython.display import clear_output
from math import ceil,floor

%matplotlib inline
```

Clase Agente

```
class PongAgent:

    def __init__(self, game, policy=None, discount_factor = 0.1, learning_rate

        # Creamos la tabla de politicas
        if policy is not None:
            self._q_table = policy
        else:
            position = list(game.positions_space.shape)
            position.append(len(game.action_space))
            self._q_table = np.zeros(position)

        self.discount_factor = discount_factor
        self.learning_rate = learning_rate
        self.ratio_explotacion = ratio_explotacion

    def get_next_step(self, state, game):

        # Damos un paso aleatorio...
        next_step = np.random.choice(list(game.action_space))

        # o tomaremos el mejor paso...
        if np.random.uniform() <= self.ratio_explotacion:
            # tomar el maximo
            idx_action = np.random.choice(np.flatnonzero(
                    self._q_table[state[0],state[1],state[2]] == self._q_table[
                ))
            next_step = list(game.action_space)[idx_action]

        return next_step

    # actualizamos las politicas con las recompensas obtenidas
    def update(self, game, old_state, action_taken, reward_action_taken, new_st
        idx_action_taken =list(game.action_space).index(action_taken)

        actual_q_value_options = self._q_table[old_state[0], old_state[1], old_
        actual_q_value = actual_q_value_options[idx_action_taken]

        future_q_value_options = self._q_table[new_state[0], new_state[1], new_
        future_max_q_value = reward_action_taken  +  self.discount_factor*futur
        if reached_end:
            future_max_q_value = reward_action_taken #maximum reward

        self._q_table[old_state[0], old_state[1], old_state[2], idx_action_take
                                                self.learning_rate*(future_max_q_
```

```
    def print_policy(self):
        for row in np.round(self._q_table,1):
            for column in row:
                print('[', end='')
                for value in column:
                    print(str(value).zfill(5), end=' ')
                print('] ', end='')
            print('')

    def get_policy(self):
        return self._q_table
```

Clase Environment

```
class PongEnvironment:

    def __init__(self, max_life=3, height_px = 40, width_px = 50, movimiento_px

        self.action_space = ['Arriba','Abajo']

        self._step_penalization = 0

        self.state = [0,0,0]

        self.total_reward = 0

        self.dx = movimiento_px
        self.dy = movimiento_px

        filas = ceil(height_px/movimiento_px)
        columnas = ceil(width_px/movimiento_px)

        self.positions_space = np.array([[[0 for z in range(columnas)]
                                                for y in range(filas)]
                                                   for x in range(filas)])


        self.lives = max_life
        self.max_life=max_life

        self.x = randint(int(width_px/2), width_px)
        self.y = randint(0, height_px-10)

        self.player_alto = int(height_px/4)

        self.player1 = self.player_alto  # posic. inicial del player

        self.score = 0
```

```
        self.width_px = width_px
        self.height_px = height_px
        self.radio = 2.5

    def reset(self):
        self.total_reward = 0
        self.state = [0,0,0]
        self.lives = self.max_life
        self.score = 0
        self.x = randint(int(self.width_px/2), self.width_px)
        self.y = randint(0, self.height_px-10)
        return self.state

    def step(self, action, animate=False):
        self._apply_action(action, animate)
        done = self.lives <=0 # final
        reward = self.score
        reward += self._step_penalization
        self.total_reward += reward
        return self.state, reward , done

    def _apply_action(self, action, animate=False):

        if action == "Arriba":
            self.player1 += abs(self.dy)
        elif action == "Abajo":
            self.player1 -= abs(self.dy)

        self.avanza_player()

        self.avanza_frame()

        if animate:
            clear_output(wait=True);
            fig = self.dibujar_frame()
            plt.show()

        self.state = (floor(self.player1/abs(self.dy))-2, floor(self.y/abs(self

    def detectaColision(self, ball_y, player_y):
        if (player_y+self.player_alto >= (ball_y-self.radio)) and (player_y <=
            return True
        else:
            return False

    def avanza_player(self):
        if self.player1 + self.player_alto >= self.height_px:
            self.player1 = self.height_px - self.player_alto
        elif self.player1 <= -abs(self.dy):
            self.player1 = -abs(self.dy)
```

```
        def avanza_frame(self):
            self.x += self.dx
            self.y += self.dy
            if self.x <= 3 or self.x > self.width_px:
                self.dx = -self.dx
                if self.x <= 3:
                    ret = self.detectaColision(self.y, self.player1)

                    if ret:
                        self.score = 10
                    else:
                        self.score = -10
                        self.lives -= 1
                        if self.lives>0:
                            self.x = randint(int(self.width_px/2), self.width_px)
                            self.y = randint(0, self.height_px-10)
                            self.dx = abs(self.dx)
                            self.dy = abs(self.dy)
            else:
                self.score = 0

            if self.y < 0 or self.y > self.height_px:
                self.dy = -self.dy

        def dibujar_frame(self):
            fig = plt.figure(figsize=(5, 4))
            a1 = plt.gca()
            circle = plt.Circle((self.x, self.y), self.radio, fc='slategray', ec="b
            a1.set_ylim(-5, self.height_px+5)
            a1.set_xlim(-5, self.width_px+5)

            rectangle = plt.Rectangle((-5, self.player1), 5, self.player_alto, fc='
            a1.add_patch(circle);
            a1.add_patch(rectangle)
            #a1.set_yticklabels([]);a1.set_xticklabels([]);
            plt.text(4, self.height_px, "SCORE:"+str(self.total_reward)+"  LIFE:"+s
            if self.lives <=0:
                plt.text(10, self.height_px-14, "GAME OVER", fontsize=16)
            elif self.total_reward >= 1000:
                plt.text(10, self.height_px-14, "YOU WIN!", fontsize=16)
            return fig
```

Juego

```
    def play(rounds=5000, max_life=3, discount_factor = 0.1, learning_rate = 0.1,
             ratio_explotacion=0.9,learner=None, game=None, animate=False):

        if game is None:
```

```
        # si usamos movimiento_px = 5 creamos una tabla de politicas de 8x10
        # si usamos movimiento_px = 3 la tabla sera de 14x17
        game = PongEnvironment(max_life=max_life, movimiento_px = 3)

    if learner is None:
        print("Begin new Train!")
        learner = PongAgent(game, discount_factor = discount_factor,learning_ra

    max_points= -9999
    first_max_reached = 0
    total_rw=0
    steps=[]

    for played_games in range(0, rounds):
        state = game.reset()
        reward, done = None, None

        itera=0
        while (done != True) and (itera < 3000 and game.total_reward<=1000):
            old_state = np.array(state)
            next_action = learner.get_next_step(state, game)
            state, reward, done = game.step(next_action, animate=animate)
            if rounds > 1:
                learner.update(game, old_state, next_action, reward, state, dor
            itera+=1

        steps.append(itera)

        total_rw+=game.total_reward
        if game.total_reward > max_points:
            max_points=game.total_reward
            first_max_reached = played_games

        if played_games %500==0 and played_games >1 and not animate:
            print("-- Partidas[", played_games, "] Avg.Puntos[", int(total_rw/p

    if played_games>1:
        print('Partidas[',played_games,'] Avg.Puntos[',int(total_rw/played_game

    #learner.print_policy()

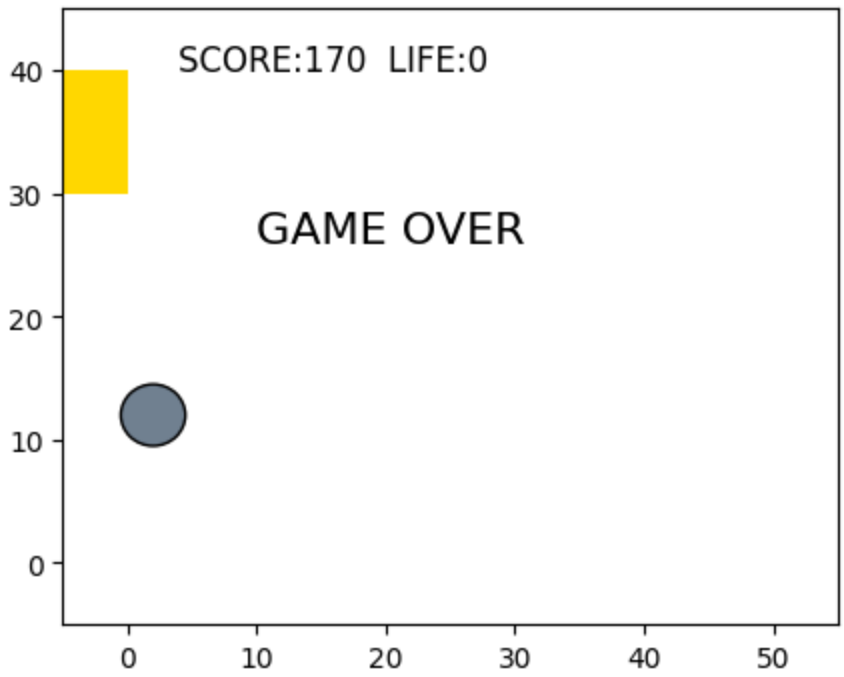
    return learner, game
```

```
learner, game = play(rounds=5000, discount_factor = 0.2, learning_rate = 0.1, r
```

```
Begin new Train!
-- Partidas[ 500 ] Avg.Puntos[ 10 ]  AVG Steps[ 205 ] Max Score[ 160 ]
-- Partidas[ 1000 ] Avg.Puntos[ 15 ]  AVG Steps[ 225 ] Max Score[ 160 ]
-- Partidas[ 1500 ] Avg.Puntos[ 17 ]  AVG Steps[ 231 ] Max Score[ 160 ]
-- Partidas[ 2000 ] Avg.Puntos[ 21 ]  AVG Steps[ 242 ] Max Score[ 230 ]
-- Partidas[ 2500 ] Avg.Puntos[ 22 ]  AVG Steps[ 248 ] Max Score[ 240 ]
```

```
-- Partidas[ 3000 ] Avg.Puntos[ 25 ]  AVG Steps[ 256 ] Max Score[ 690 ]
-- Partidas[ 3500 ] Avg.Puntos[ 26 ]  AVG Steps[ 259 ] Max Score[ 690 ]
-- Partidas[ 4000 ] Avg.Puntos[ 28 ]  AVG Steps[ 266 ] Max Score[ 690 ]
-- Partidas[ 4500 ] Avg.Puntos[ 30 ]  AVG Steps[ 272 ] Max Score[ 690 ]
Partidas[ 4999 ] Avg.Puntos[ 31 ] Max score[ 690 ] en partida[ 2520 ]
```

```
learner2 = PongAgent(game, policy=learner.get_policy())
learner2.ratio_explotacion = 1.0  # con esto quitamos las elecciones aleatorias
player = play(rounds=1, learner=learner2, game=game, animate=True)
```



Desafio

Prueba con diferentes cantidades de epoca así como con más o menos vidas. Elabora cuando hay mejor rendimiento, puedes usar las funciones de tiempo que se vieron en prácticas pasadas del laboratorio

```
import time

print("=== EXPERIMENTO DEL DESAFÍO ===")

# Configuraciones a probar: (rondas, vidas)
configuraciones = [
    (1000, 1),
    (3000, 3),
    (5000, 5)
]

for rondas, vidas in configuraciones:
```

```
        print(f"\n--- Probando con {rondas} rondas y {vidas} vidas ---")
        inicio = time.time()

        agente_test, juego_test = play(
            rounds=rondas,
            max_life=vidas,
            discount_factor=0.2,
            learning_rate=0.1,
            ratio_explotacion=0.85
        )

        fin = time.time()
        tiempo_total = fin - inicio
        print(f"Tiempo de entrenamiento: {tiempo_total:.2f} segundos")
```

```
=== EXPERIMENTO DEL DESAFÍO ===

--- Probando con 1000 rondas y 1 vidas ---
Begin new Train!
-- Partidas[ 500 ] Avg.Puntos[ -3 ]  AVG Steps[ 42 ] Max Score[ 70 ]
Partidas[ 999 ] Avg.Puntos[ 0 ] Max score[ 170 ] en partida[ 974 ]
Tiempo de entrenamiento: 3.47 segundos

--- Probando con 3000 rondas y 3 vidas ---
Begin new Train!
-- Partidas[ 500 ] Avg.Puntos[ 9 ]  AVG Steps[ 202 ] Max Score[ 240 ]
-- Partidas[ 1000 ] Avg.Puntos[ 12 ]  AVG Steps[ 213 ] Max Score[ 240 ]
-- Partidas[ 1500 ] Avg.Puntos[ 16 ]  AVG Steps[ 226 ] Max Score[ 280 ]
-- Partidas[ 2000 ] Avg.Puntos[ 20 ]  AVG Steps[ 239 ] Max Score[ 280 ]
-- Partidas[ 2500 ] Avg.Puntos[ 22 ]  AVG Steps[ 246 ] Max Score[ 280 ]
Partidas[ 2999 ] Avg.Puntos[ 25 ] Max score[ 350 ] en partida[ 2631 ]
Tiempo de entrenamiento: 45.22 segundos

--- Probando con 5000 rondas y 5 vidas ---
Begin new Train!
-- Partidas[ 500 ] Avg.Puntos[ 34 ]  AVG Steps[ 405 ] Max Score[ 240 ]
-- Partidas[ 1000 ] Avg.Puntos[ 30 ]  AVG Steps[ 391 ] Max Score[ 300 ]
-- Partidas[ 1500 ] Avg.Puntos[ 35 ]  AVG Steps[ 409 ] Max Score[ 350 ]
-- Partidas[ 2000 ] Avg.Puntos[ 44 ]  AVG Steps[ 438 ] Max Score[ 360 ]
-- Partidas[ 2500 ] Avg.Puntos[ 52 ]  AVG Steps[ 463 ] Max Score[ 400 ]
-- Partidas[ 3000 ] Avg.Puntos[ 56 ]  AVG Steps[ 478 ] Max Score[ 470 ]
-- Partidas[ 3500 ] Avg.Puntos[ 59 ]  AVG Steps[ 489 ] Max Score[ 470 ]
-- Partidas[ 4000 ] Avg.Puntos[ 62 ]  AVG Steps[ 498 ] Max Score[ 470 ]
-- Partidas[ 4500 ] Avg.Puntos[ 65 ]  AVG Steps[ 508 ] Max Score[ 530 ]
Partidas[ 4999 ] Avg.Puntos[ 66 ] Max score[ 530 ] en partida[ 4458 ]
Tiempo de entrenamiento: 146.66 segundos
```

Conclusiones finales del estudiante

En esta práctica fue muy entretenido ver en acción el aprendizaje por refuerzo aplicado a un juego clásico como Pong. Me gustó mucho observar cómo el agente empieza

completamente a ciegas, fallando casi todos los tiros, y con solo repetir partidas y ajustar sus recompensas termina aprendiendo por su cuenta a colocarse justo donde va a rebotar la pelota. Es una forma muy visual y divertida de entender cómo una máquina puede aprender de sus propios aciertos y errores sin necesidad de que le programemos reglas fijas para cada movimiento.

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados